Análise de Dados

dr. Rachid Muleia, Msc. in Statistics

Universidade Eduardo Mondlane
Faculdade de Ciências
Departamento de Matemática e Informática

Julho - Novembro 2019

Avaliação da qualidade de dados (DQA)

- O objectivo primário é determinar se uma decisão ou estimativa pode ser feita com o nível de certeza desejado, dado a qualidade dos dados
- Verificar a adequação das variáveis de interesse.
- Verificar a adequação dos instrumentos de medição.
- Suficiência da amostra para detectar um efeito com significância prática caso esteja presente.

Revisão de dados

Durante o processo de revisão de dados é preciso inspeccionar os dados na sua totalidade, gerando uma série de estatísticas sumárias e visualizações gráficas.

- Calcular o mínimo e máximo para todas as variáveis, e comparar-os com os intervalos predeterminados.
- Eliminar duplicados da base de dados.
- verificar se os dados foram colhidos na unidade de medida correcta.
- calcular a correlação serial para verificar se as observações são independentes umas das outra (correlação entre a variável e os seus valores defasados)

O que é R?

- Uma linguagem de programação.
- Software para análises estatísticas e visualização de dados.
- Open source *não precisa de gastar algum centavo*
- Está disponível para os sistemas operativos: Linux, Mac e Windows.
- Pode ser baixado a partir do repositório CRAN ou The R Project for Statistical Computing
- Recomendo que também instale o RStudio, um editor que ajuda na aprendizagem e garante maior produtividade.

Por que usar R?

- Actualmente é uma das liguangens de programação com maior popularidade.
- R é bastante recomendável para Data Science.
- Tem uma imensidão de recursos para aprendizagem.
- Muita gente empenhada no desenvolvimento do R
- A comunidade de usuários de R é bastante grande. *Check* Stackoverflow R
- "Melhor software para visualização de dados."
- Desenvolvimento de aplicações webs, vulgarmente conhecidas por Shiny App

Instalação e Inicialização

O processo de instalação é similar ao de qualquer programa.

- O ficheiro é executavel, então é só clicar duas vezes e seguir as instruções
- Após a instalação o programa pode ser incializado indo ao Sart Menu e procurar pelo icon:

Para maior desempenho na aprendizagem recomendo que use o Rstudio (preferência pessoal). Existem outros editores...

Operadores aritiméticos

O R também pode ser usado como calculadora.

```
x + y #adição
x - y #subtração
x * y #multiplicação
x / y #divisão
x ^ y #exponenciação
x %% y #modulus(resto da divisão)
x %/% y #divisão inteira
```

Para além de operações artiméticas, o R pode fazer operações matemáticas complexas. [Vide documentação] em https://stat.ethz.ch/R-manual/R-devel/library/base/html/Trig.html

Tipo de Dados e Estrutura de Dados

Tipo de dados

- Logical (TRUE e FALSE)
- Interger (número inteiro) ex: 1L,2L,3L...
- Numeric (número real) ex: 2.0, 4.7,...
- character (carácter/string) ex: " Global".

Estrutura de dados

- Vectors (vectores)
- Lists (listas)
- Matrix (matrizes)
- Array
- Dataframe

vector

```
apple <- c('red','green',"yellow")
print(apple)
```

```
## [1] "red"    "green"  "yellow"
```

```
#elementos do vector podem ser acedidos usando []
apple[1]
```

```
## [1] "red"
```

```
apple[2]
```

```
## [1] "green"
```

```
apple[c(1,3)]
```

```
## [1] "red"    "yellow"
```

Listas

```r
# Criacao de lista.
list1<-list(c(2,5,3),21.3,sin)
# imprimir a lista.
print(list1)
```

```
## [[1]]
## [1] 2 5 3
##
## [[2]]
## [1] 21.3
##
## [[3]]
## function (x)  .Primitive("sin")
```

Matrizes

Uma matriz é um conjunto de dados retangular bidimensional. A função `matrix()` recebe um vector.

```
# Criacao da matriz.
M = matrix( c('a','a','b','c','b','a'), nrow = 2, ncol = 3, byrow = TRUE)
print(M)
```

```
##      [,1] [,2] [,3]
## [1,] "a"  "a"  "b"
## [2,] "c"  "b"  "a"
```

```
# Os valores da matriz podem ser acedidos usando [,]
M[1,3]; M[1,1:3]
```

```
## [1] "b"
```

```
## [1] "a" "a" "b"
```

Matrizes (um pouco mais)

```
Y=matrix(1:12, nrow = 3, ncol = 4)
Y
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    4    7   10
## [2,]    2    5    8   11
## [3,]    3    6    9   12
```

```
Y[1, 3]; Y[1, ]; Y[, 2]
```

```
## [1] 7
```

```
## [1]  1  4  7 10
```

```
## [1] 4 5 6
```

Data frame

Um objecto data.frame em R tem propriedades dimensionais semelhantes a uma matriz, mas pode conter dados categóricos, além de numéricos. Uma matriz só pode levar dados do mesmo tipo.

Uma data frame pode ser criada usando a função `data.frame()`.

```
nomes=c("Edson", "Eudson","Euclides", "Freid")
idade=c(30,35,32,34)
residencia=c('Zimpeto','Liberdade','Alto-Mae', "Polana")
#data frame
dados_trab=data.frame(nomes, idade, residencia, stringsAsFactors=FALSE)
#inspeccionar
head(dados_trab)
```

```
##      nomes idade residencia
## 1    Edson    30    Zimpeto
## 2   Eudson    35  Liberdade
## 3 Euclides    32   Alto-Mae
## 4    Freid    34     Polana
```

```r
dim(dados_trab) #dimensão da base de dados.
```

```
## [1] 4 3
```

```r
names(dados_trab) # nome das variáveis.
```

```
## [1] "nomes"       "idade"       "residencia"
```

```r
str(dados_trab) # estrutura da base de dados.
```

```
## 'data.frame':    4 obs. of  3 variables:
##  $ nomes     : chr  "Edson" "Eudson" "Euclides" "Freid"
##  $ idade     : num  30 35 32 34
##  $ residencia: chr  "Zimpeto" "Liberdade" "Alto-Mae" "Polar
```

Como aceder aos elementos da Data Frame?

- Os elementos de uma data frame podem ser acedidos da mesma forma como elementos de uma lista ou matriz.

Acedendo como lista:

- Pode-se usar os seguintes operados, [, [[or $ para aceder as colunas.

```
dados_trab['nomes']
```

```
##      nomes
## 1    Edson
## 2   Eudson
## 3 Euclides
## 4    Freid
```

```
dados_trab$nomes
```

```
## [1] "Edson"    "Eudson"   "Euclides" "Freid"
```

```
dados_trab[['nomes']]
```

```
## [1] "Edson"    "Eudson"   "Euclides" "Freid"
```

Aceder como uma matriz.

```
dados_trab[1:3,] #seleccione as três primeiras linhas.
```

```
##      nomes idade residencia
## 1    Edson    30    Zimpeto
## 2   Eudson    35  Liberdade
## 3 Euclides    32   Alto-Mae
```

```
dados_trab[dados_trab$idade<34,] #seleccione os que tem idade
```

```
##      nomes idade residencia
## 1    Edson    30    Zimpeto
## 3 Euclides    32   Alto-Mae
```

```r
dados_trab[,2:3] #seleccione a segunda e a terceira coluna.
```

```
##   idade residencia
## 1    30    Zimpeto
## 2    35  Liberdade
## 3    32   Alto-Mae
## 4    34     Polana
```

Modificar elementos da Data Frame (observações).

```
dados_trab[1, 'idade']=35
dados_trab
```

```
##      nomes idade residencia
## 1    Edson    35    Zimpeto
## 2   Eudson    35  Liberdade
## 3 Euclides    32   Alto-Mae
## 4    Freid    34     Polana
```

R tem um editor de dados, no estilo de planilha, que pode ser usado para inserir os dados.

```
newdata=edit(data.frame())
dados_trab=edit(dados_trab)
```

- O R não foi concebido para inserção de dados.
- Dados são geralmente armazenados em ficheiros `.txt`, `.csv`, `.xlsx`, `.dat`, ...
- O R tem várias funções para ler vários ficheiros. A função mãe é `read.table()`

```
path="C:/Users/Rachid/Box Sync/My Box Docs/Analise de dados em
carros=read.table(paste(path,"cars.csv",sep='/'),
                  sep=',',header=TRUE)
```

O R não só pode ler dados armazenados localmente, mas também pode ler dados a partir da internet.

```
data=read.table("http://www.sthda.com/upload/decathlon.txt",
                sep='\t', header=TRUE)
head(data)
```

- Usar uma `url` na função `read.table()` não é aconselhável, pois não é "reprodutível".
- O ideal é baixar os dados e ler-os localmente.
- A partir do R é possível baixar dados/ficheiros online usando a função `download.file()`

```
# baixar um ficheiro .txt
download.file("http://www.sthda.com/upload/decathlon.txt",destfile=paste(path,"decathlon.txt",sep='/'))
```

- A função `download.file()`, não só pode ser usada para baixar ficheiros `.txt`.

```
download.file("http://www2.ntia.doc.gov/files/broadband-data/AK-NBM-CSV-Dec-2012.zip",
              destfile=paste(path,"AK-NBM-CSV-Dec-2012.zip",sep="/"))

unzip("AK-NBM-CSV-Dec-2012.zip")
data = read.delim(paste(path,"AK-NBM-Wireless-CSV-DEC-2012.TXT",sep="/"), sep = "|")
```

Exportação de Dados

- O R permite exportar/gravar dados em vários formatos.
- Um dos formatos mais populares é o `.csv`, mas usuário está livre de exportar para o formato que lhe convem.
- Para exportação de dados usa-se a função `write.table()`

```
write.table(dados_trab, file=paste(path, "dados_trab.csv",sep="/"), sep=",",row.names=FALSE)
```

- A função `write.table()` é mais genérica. Existem função mais específicas, como `write.csv()`

- `write.csv()` grava/exporta um ficheiro no formato `.csv`.

```
write.csv(dados_trab, file=paste(path, "dados_trab_1.csv",sep='/'))
```

Um pouco mais sobre Import/Export (Dados)

As funções `read.table()` e `write.table()` são funções internas com algumas litimações.

- Há ficheiros que não é possível ler/ gravar com estas funções.
- Por ex, ficheiros em formato `.xlsx` não podem ser lidos usando a função `read.table()`

```
#install.packages("openxlsx")
library(openxlsx)
example_xlsx=openxlsx::read.xlsx(xlsxFile = paste(path,
                                                  "data_example.xlsx", sep="/"))
head(example_xlsx)
```

A função `read.xlsx()` permite também especificar a planilha de dados que se pretende ler usando o argumento `sheet`.

O argumento `sheet` pode ser o nome da panilha, em aspas, ou o nr da planilha.

```
library(openxlsx)
example_xlsx_1=openxlsx::read.xlsx(xlsxFile = paste(path, "data_example.xlsx"
                                                    , sep="/"),sheet=2)
dim(example_xlsx)
dim(example_xlsx_1)
```

Importação de ficheiros SPSS, SAS e STATA

O R oferece recursos para leitura de ficheiros SPSS, SAS e STATA, a partir da livraria `foreign`.

Caso a livraria `foreign` não esteja instalada, pode-se instalar usando o condamando `install.packages('foreign')`.

```
#install.packages("foreign")
library(foreign)
#leitura de ficheiro SPSS
basedados=read.spss('C:/Users/lucp8943/Dropbox/Analise de Dados DMI/Slides Aulas/MZAR72FL.SAV',
                    to.data.frame=TRUE)
```

```
## re-encoding from UTF-8

## Warning in read.spss("C:/Users/lucp8943/Dropbox/Analise de Dados DMI/
## Slides Aulas/MZAR72FL.SAV", : Undeclared level(s) 577, 624, 643, 651, 669,
## 714, 739, 745, 746, 760, 763, 779, 800, 808, 810, 834, 852, 858, 859, 889,
## 901, 914, 927, 934, 940, 954, 960, 967, 969, 974, 981, 985, 1002, 1009,
## 1019, 1024, 1031, 1036, 1067, 1076, 1080, 1083, 1091, 1099, 1117, 1119,
## 1130, 1141, 1154, 1163, 1199, 1218, 1222, 1248, 1275, 1284, 1285, 1293,
## 1302, 1326, 1360, 1368, 1378, 1394, 1401, 1407, 1417, 1427, 1448, 1458,
## 1472, 1476, 1479, 1489, 1500, 1511, 1532, 1543, 1545, 1559, 1565, 1587,
## 1595, 1599, 1611, 1622, 1633, 1643, 1656, 1668, 1692, 1704, 1711, 1716,
## 1735, 1760, 1765, 1778, 1803, 1808, 1816, 1828, 1829, 1842, 1855, 1866,
## 1881, 1935, 1949, 2005, 2033, 2048, 2062, 2107, 2121, 2137, 2167, 2182,
## 2198, 2213, 2229, 2239, 2283, 2293, 2295, 2319, 2326, 2342, 2393, 2427,
## 2444, 2461, 2479, 2500, 2514, 2532, 2547, 2587, 2597, 2642, 2679, 2698,
## 2718, 2736, 2737, 2756, 2796, 2811, 2816, 2876, 2913, 2916, 2938, 2958,
## 2962, 2979, 2980, 3001, 3013, 3022, 3043, 3055, 3065, 3086, 3087, 3109,
## 3135, 3151, 3153, 3175, 3198, 3201, 3290, 3313, 3337, 3360, 3397, 3408
```

- Para além da livraria ‘foreign‘, pode-se usar a livraria ‘haven‘ para os mesmos fins.
- A livraria ‘haven‘ pode ser facilmente instalada usando o comando ‘install.packages(’haven’)‘.
- A livraria ‘haven‘ é mais recomendada e preferida para leitura de ficheiros SPSS. Além ficheiros SPSS, pode ler ficheiros SAS e STATA.

```
#install.packages('haven')
library(haven)
basedados_1=read_spss('C:/Users/lucp8943/Dropbox/Analise de Dados DMI/Slides Aulas/MZAR72FL.SAV')
```

Como podem ver, `read_spss` devolve uma `tibble`. Caso queira converter para uma `data frame`, é só usar o comando `as.data.frame()`.

```
basedados_convr=as.data.frame(basedados_1)
head(basedados_convr)
```

```
##   HIVCLUST HIVNUMB HIVLINE HIV01 HIV02 HIV03   HIV05 HIV06
## 1        1       2       1 K8L2G  6918     0 1725733     0
## 2        1       2       2 H4B4Q  6881     0 1735684     0
## 3        1      11       2 P2E5B  6950     0 1725733     0
## 4        1      12       2 G0P7D  6897     0 1725733     0
## 5        1      12       1 G8P6K  6851     0 1735684     1
## 6        1      12       6 M0R9N  2749     0 1708158    NA
##   HIV09 SHIV50 SLAGRECN SVIRAL SBIOARV SLAGE
## 1    NA     NA       NA     NA      NA     1
## 2    NA     NA       NA     NA      NA     1
## 3    NA     NA       NA     NA      NA     1
## 4    NA     NA       NA     NA      NA     1
## 5    NA     NA       NA     NA      NA     1
## 6    NA     NA       NA     NA      NA     3
```

O comando `read_spss()`, da livraria `haven`, permite aceder facilmente as *labels* dos nomes das variáveis.

```
attr(basedados_1[['HIV03']],which='label')
```

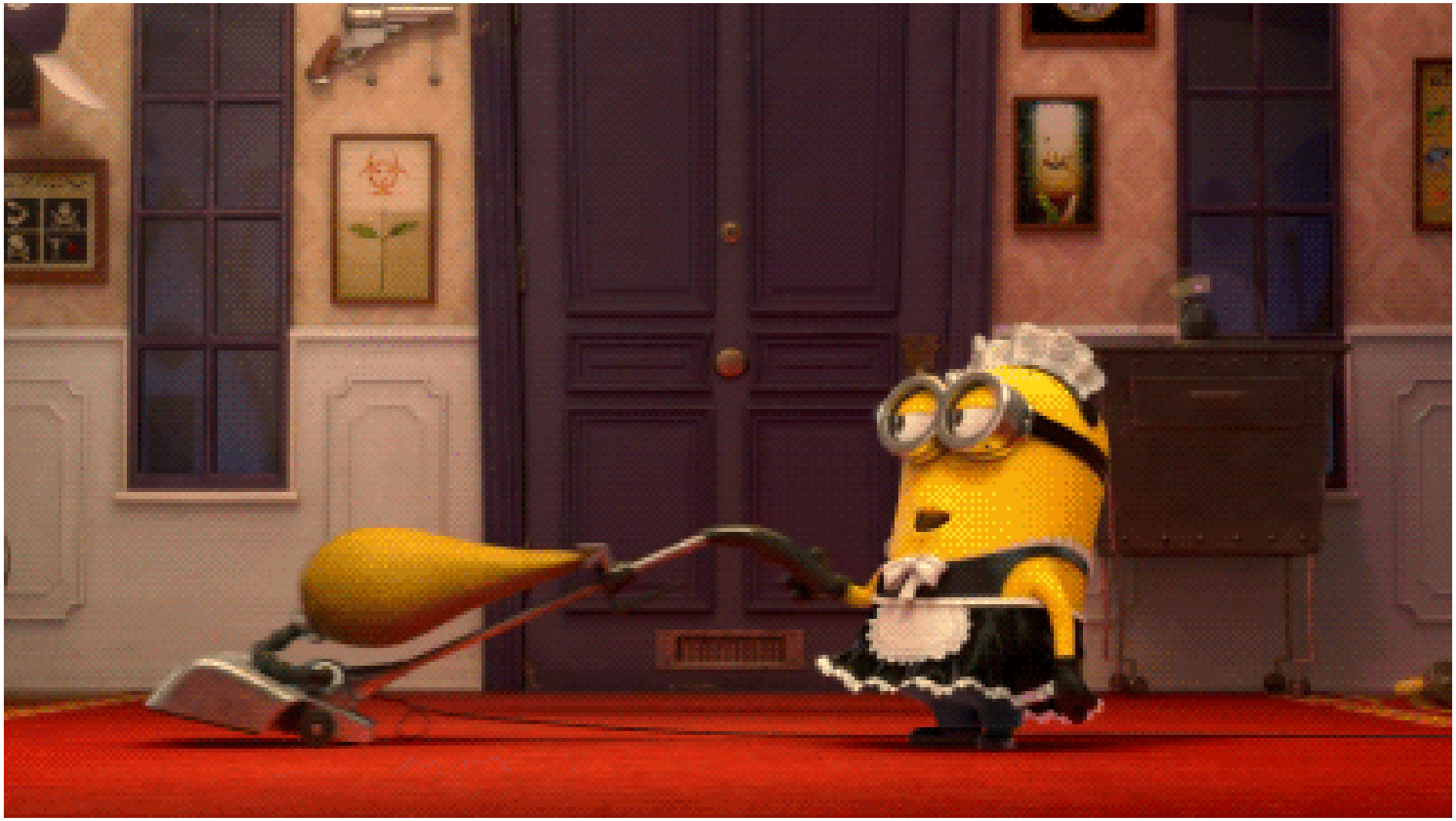
```
## [1] "Blood test result"
```

```
attr(basedados_1[['HIVNUMB']],which='label')
```

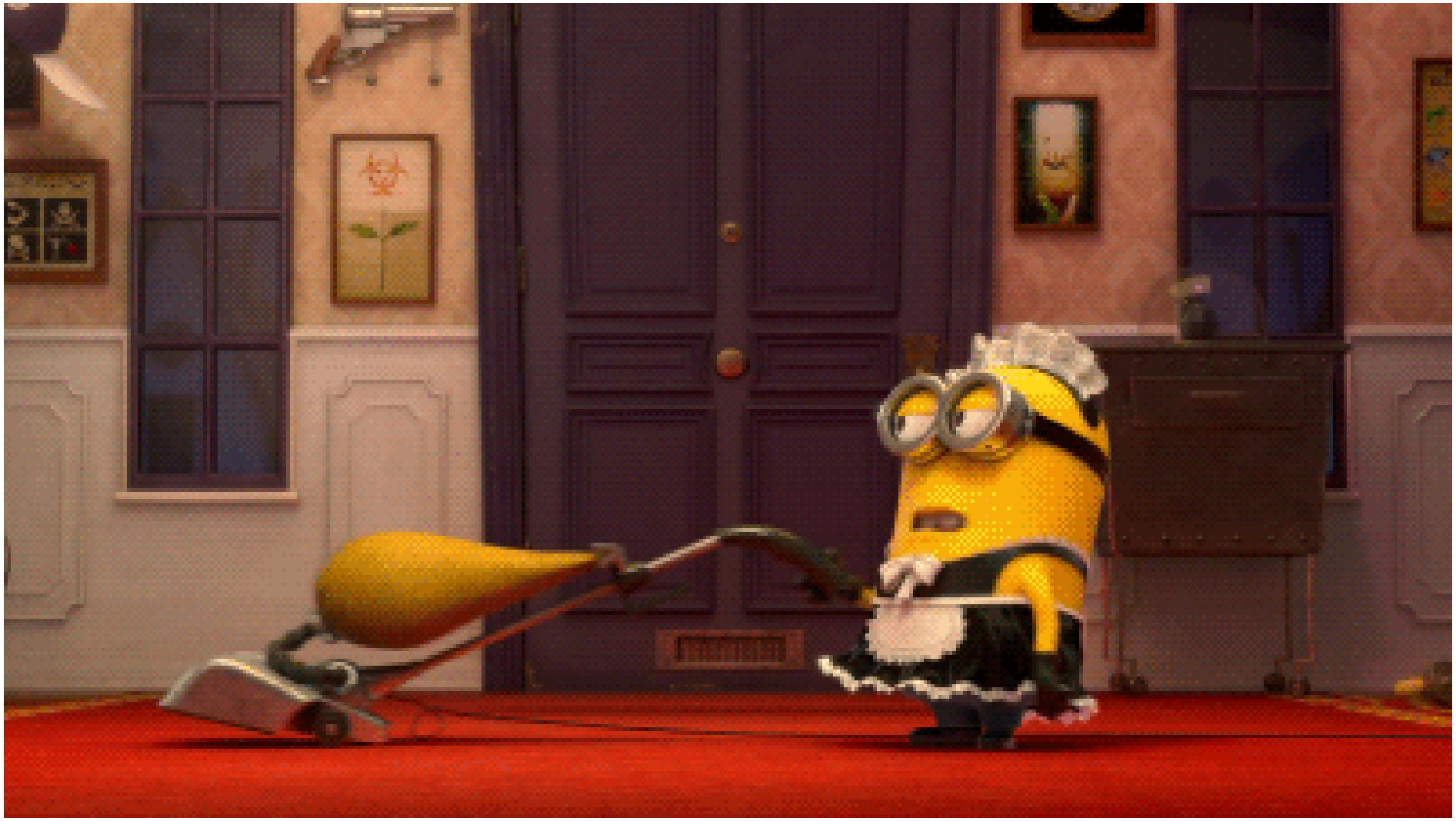
```
## [1] "Household"
```

Manipulação de dados

Manipulação de dados



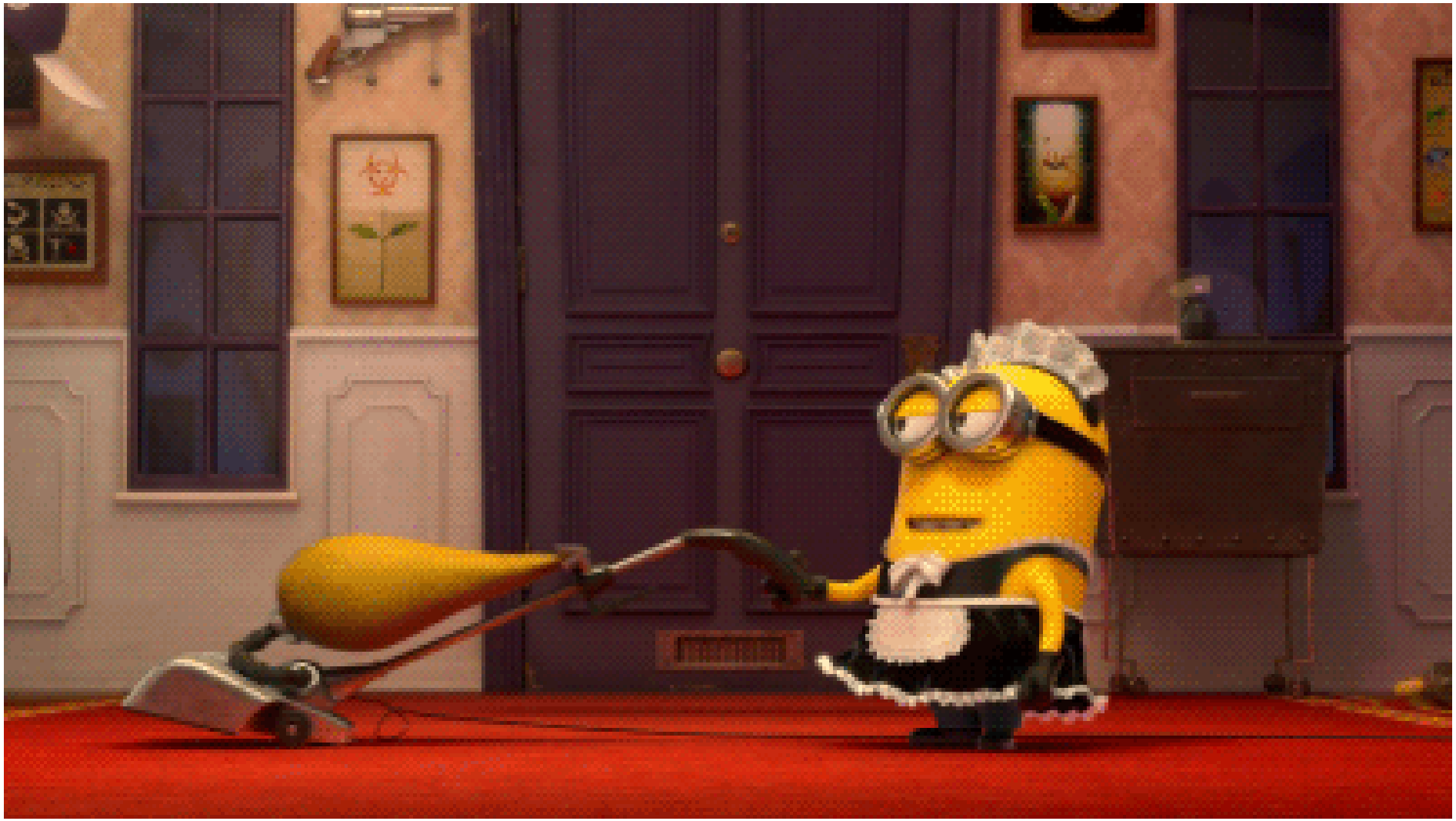
Manipulação de dados



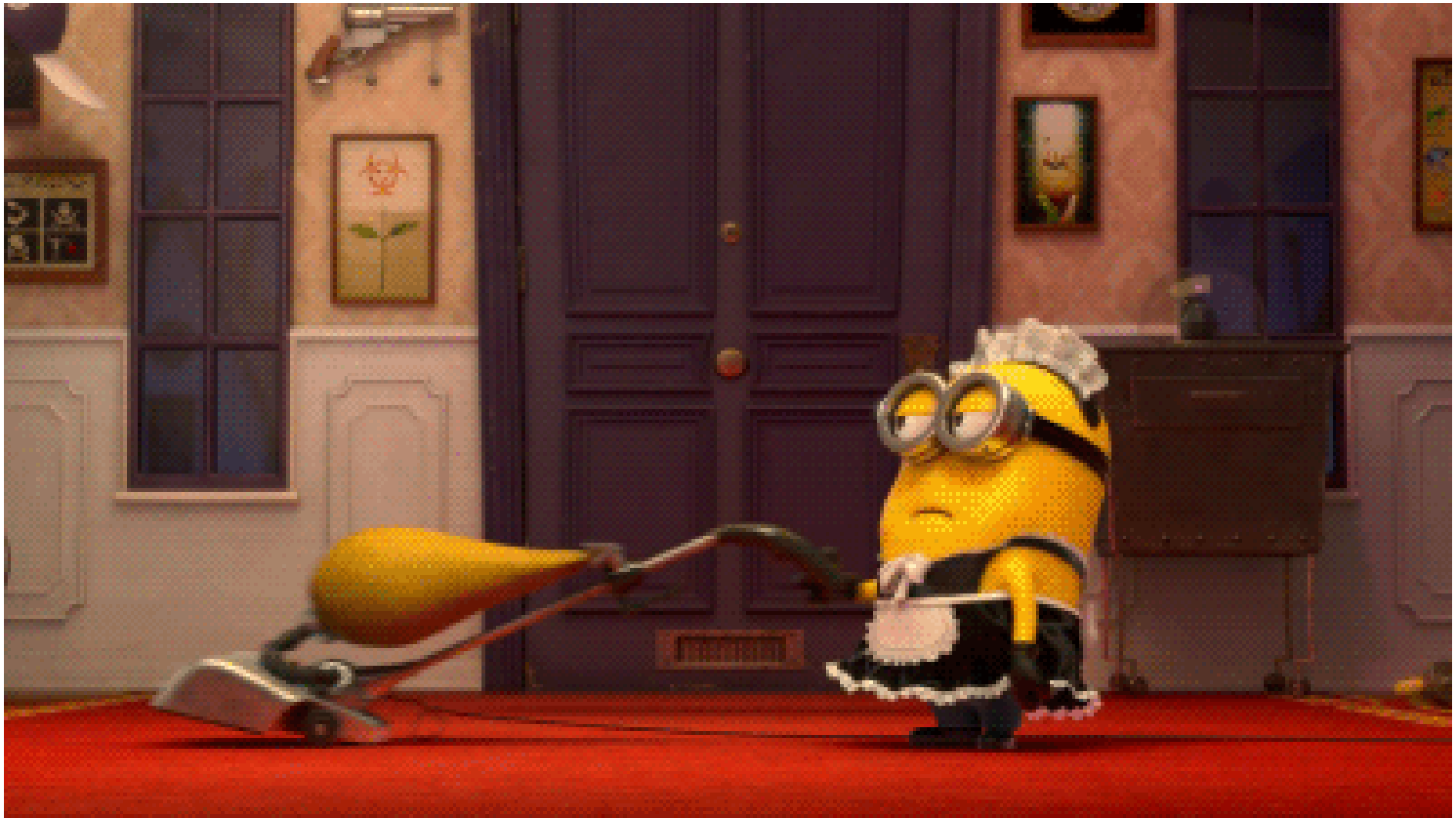
Manipulação de dados

Manipulação de dados

Manipulação de dados

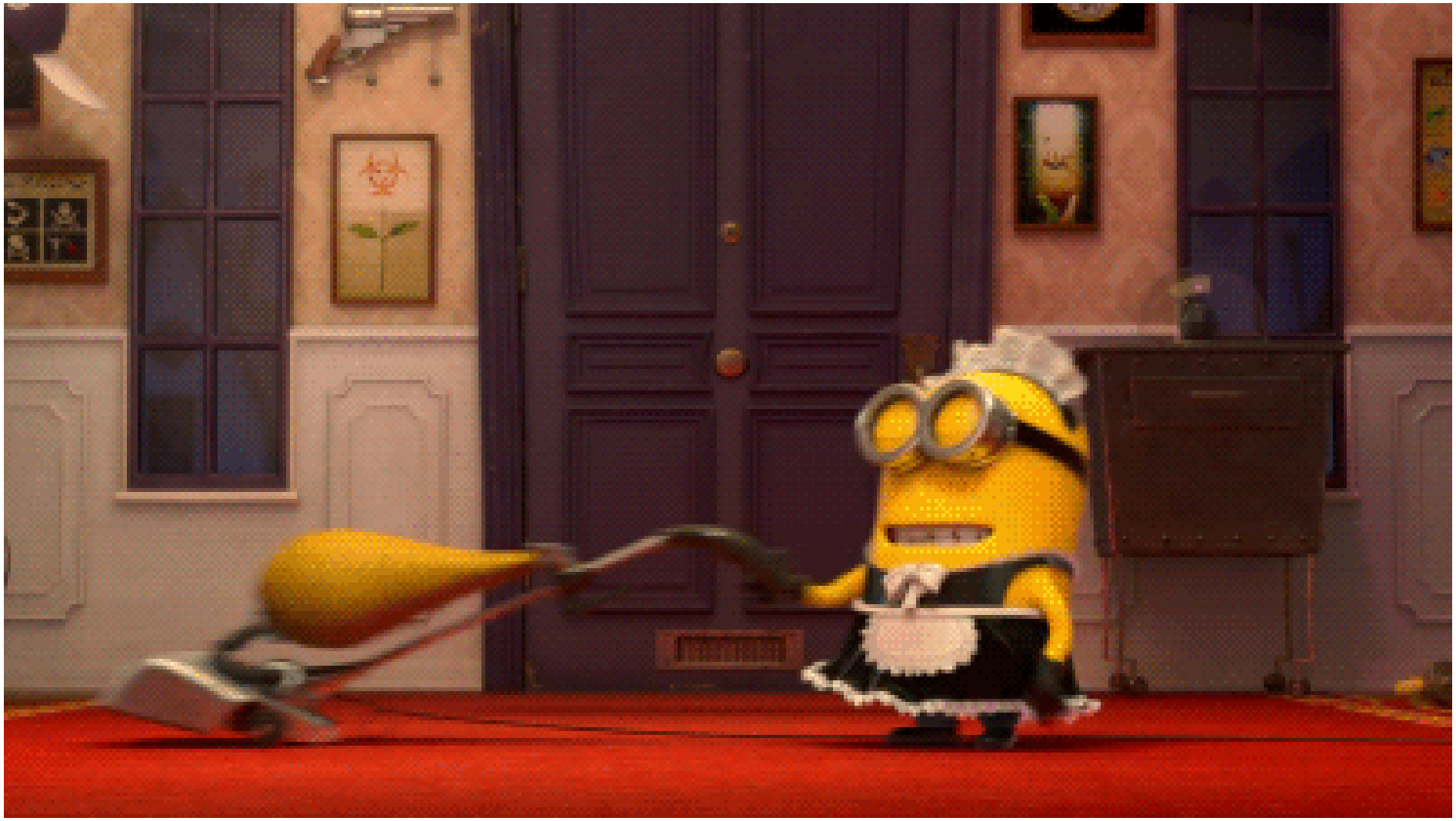


Manipulação de dados



Manipulação de dados

Manipulação de dados



Manipulação de dados

Manipulação de dados

Manipulação de dados

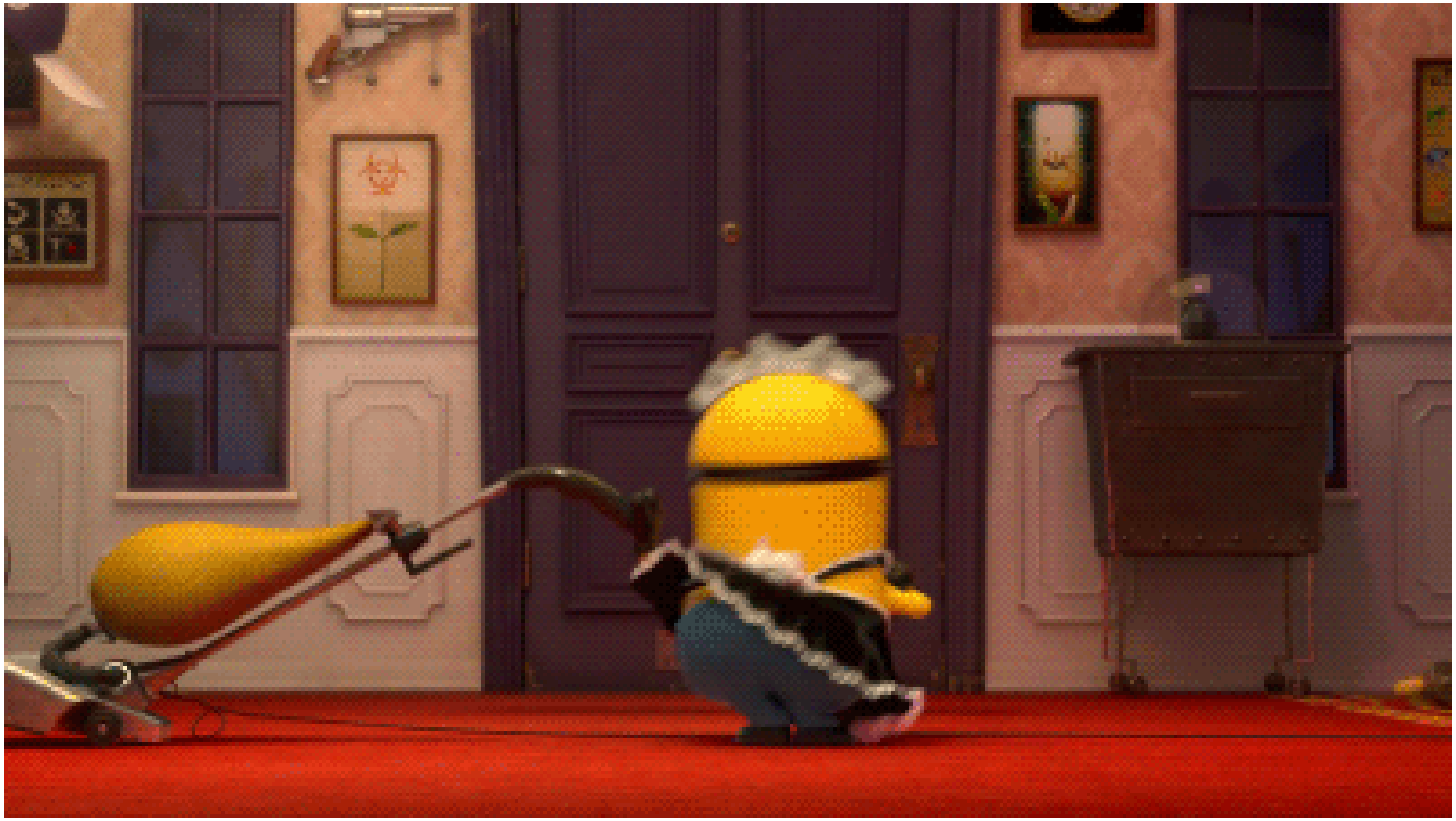
Manipulação de dados

Manipulação de dados

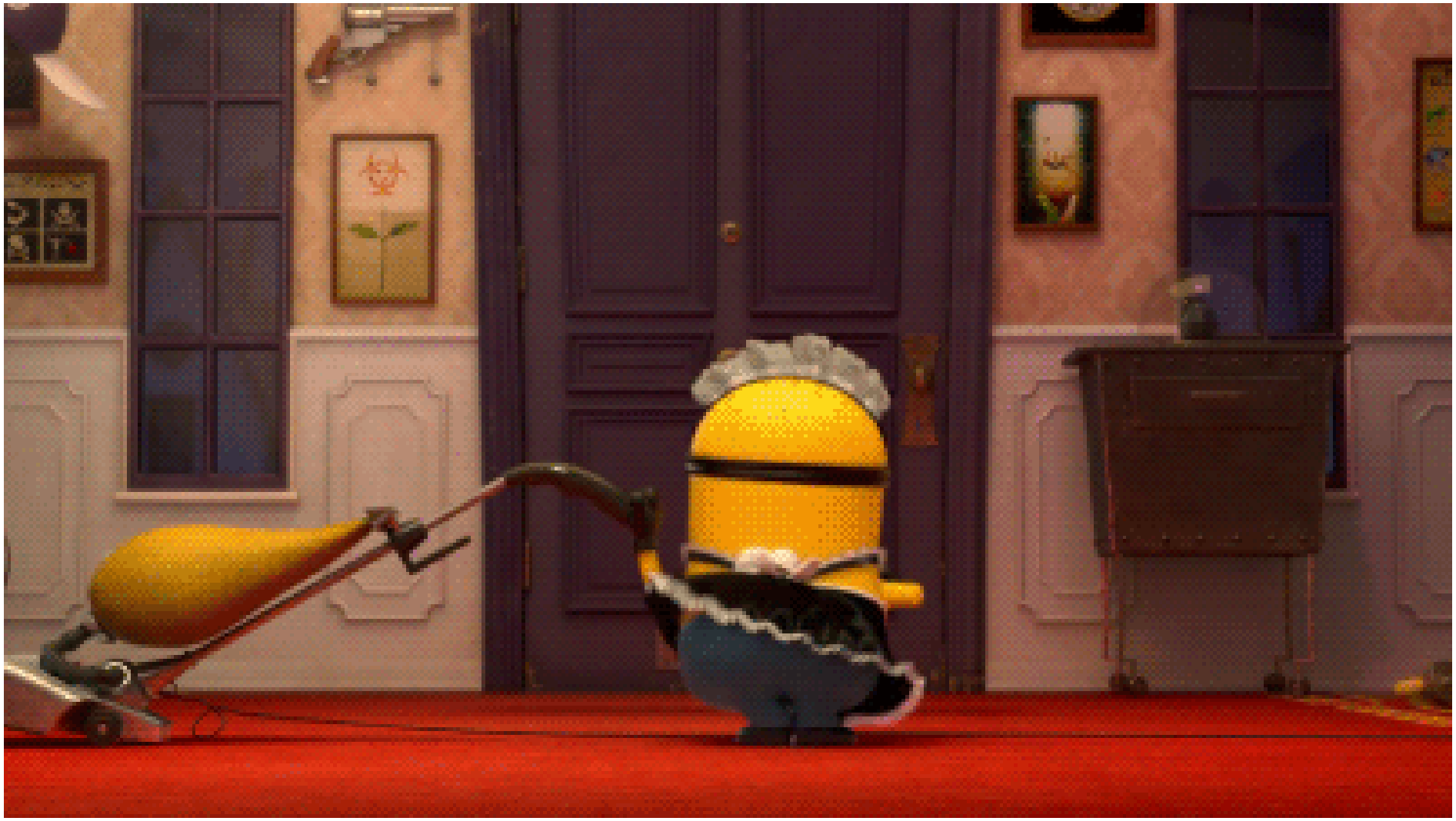
Manipulação de dados

Manipulação de dados

Manipulação de dados



Manipulação de dados

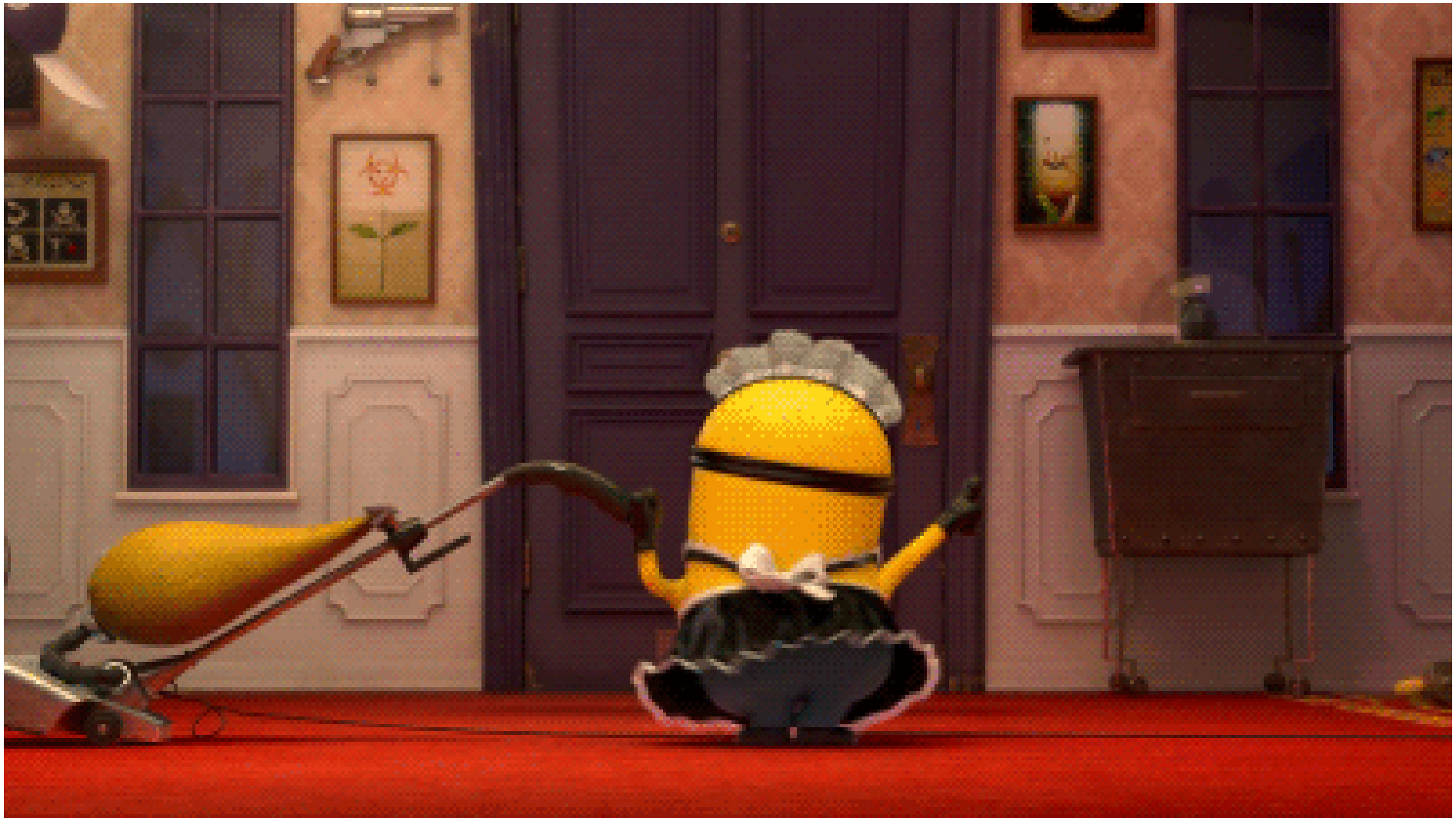


Manipulação de dados

Manipulação de dados

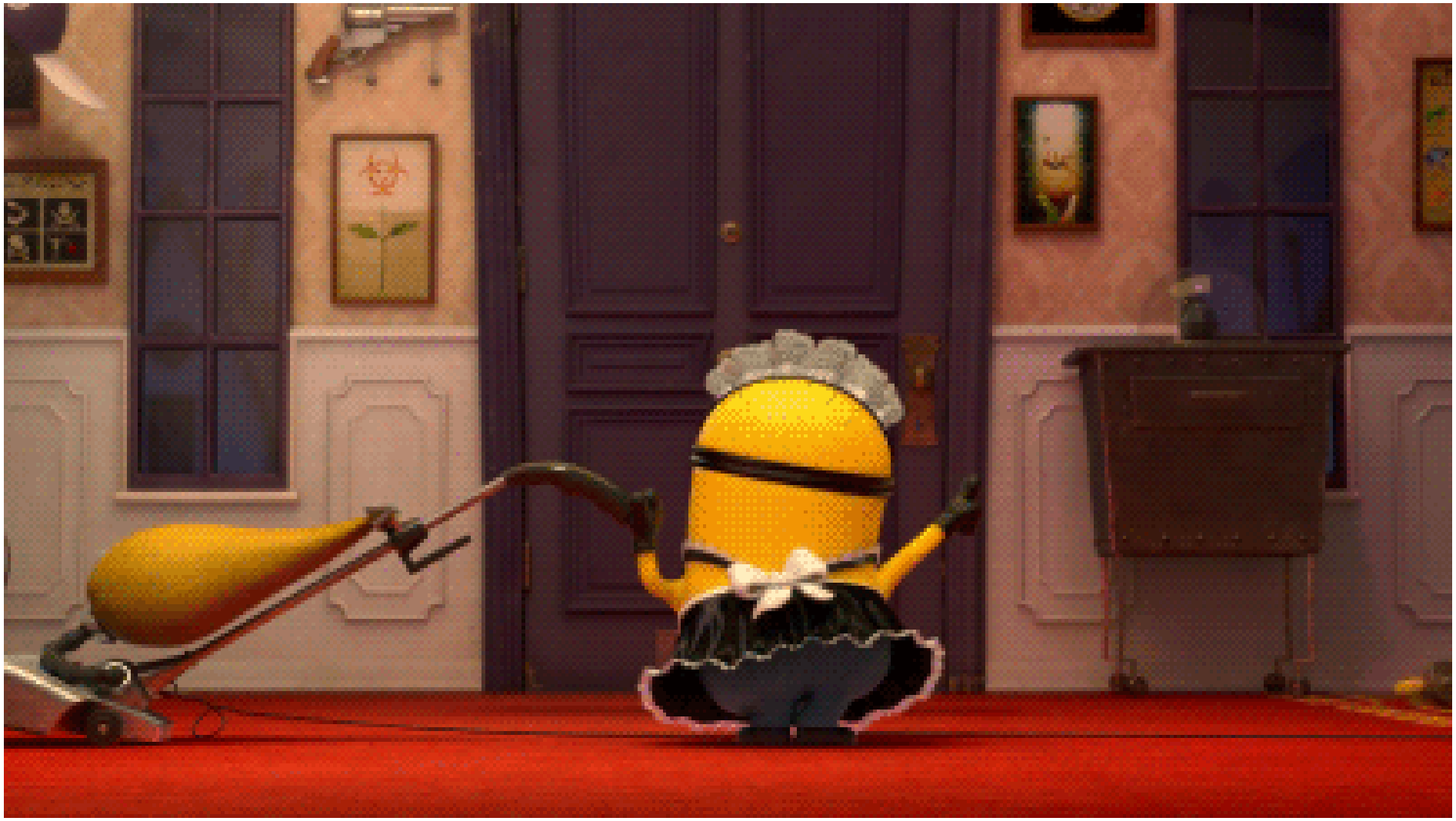
Manipulação de dados

Manipulação de dados

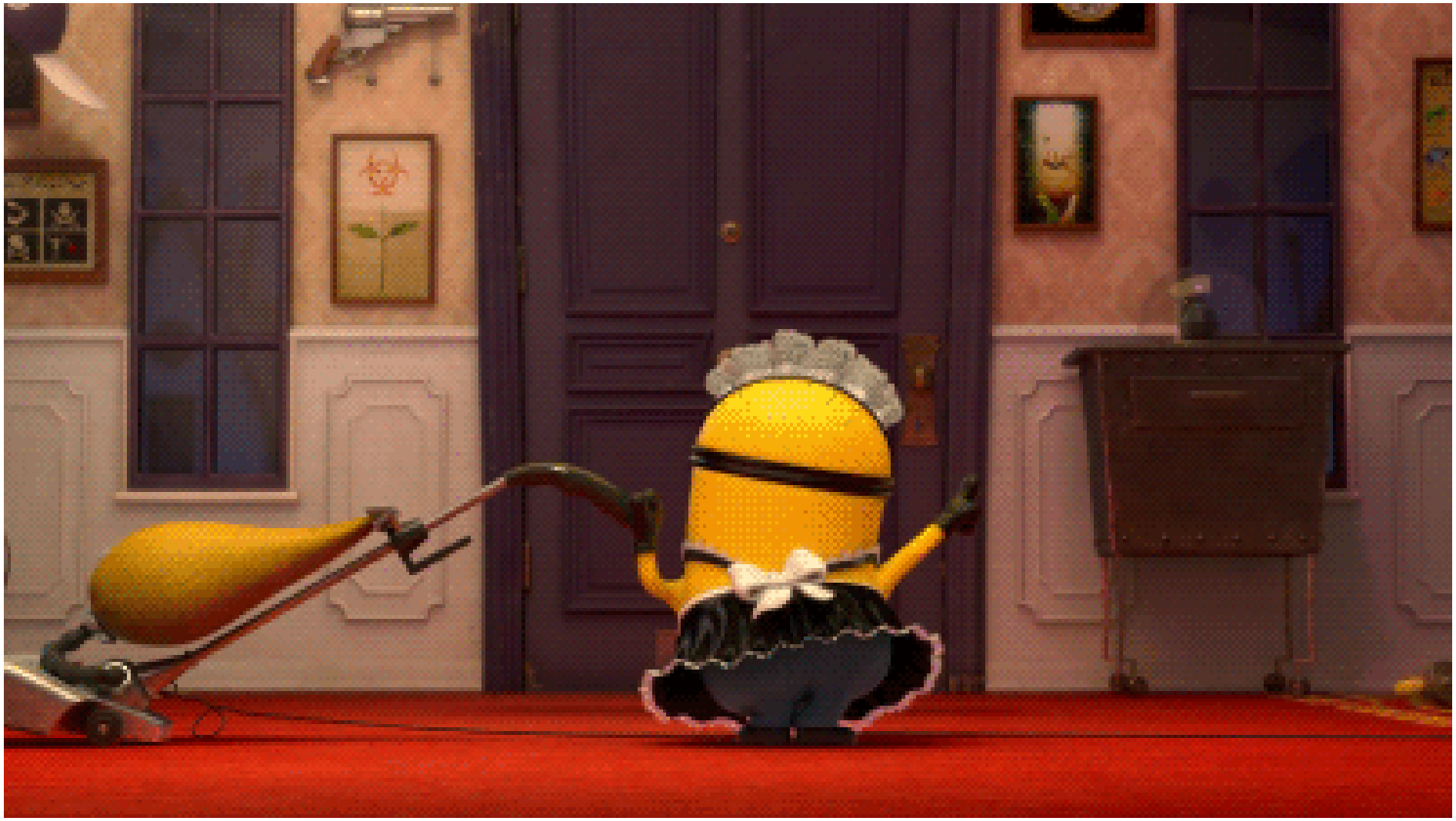


Manipulação de dados

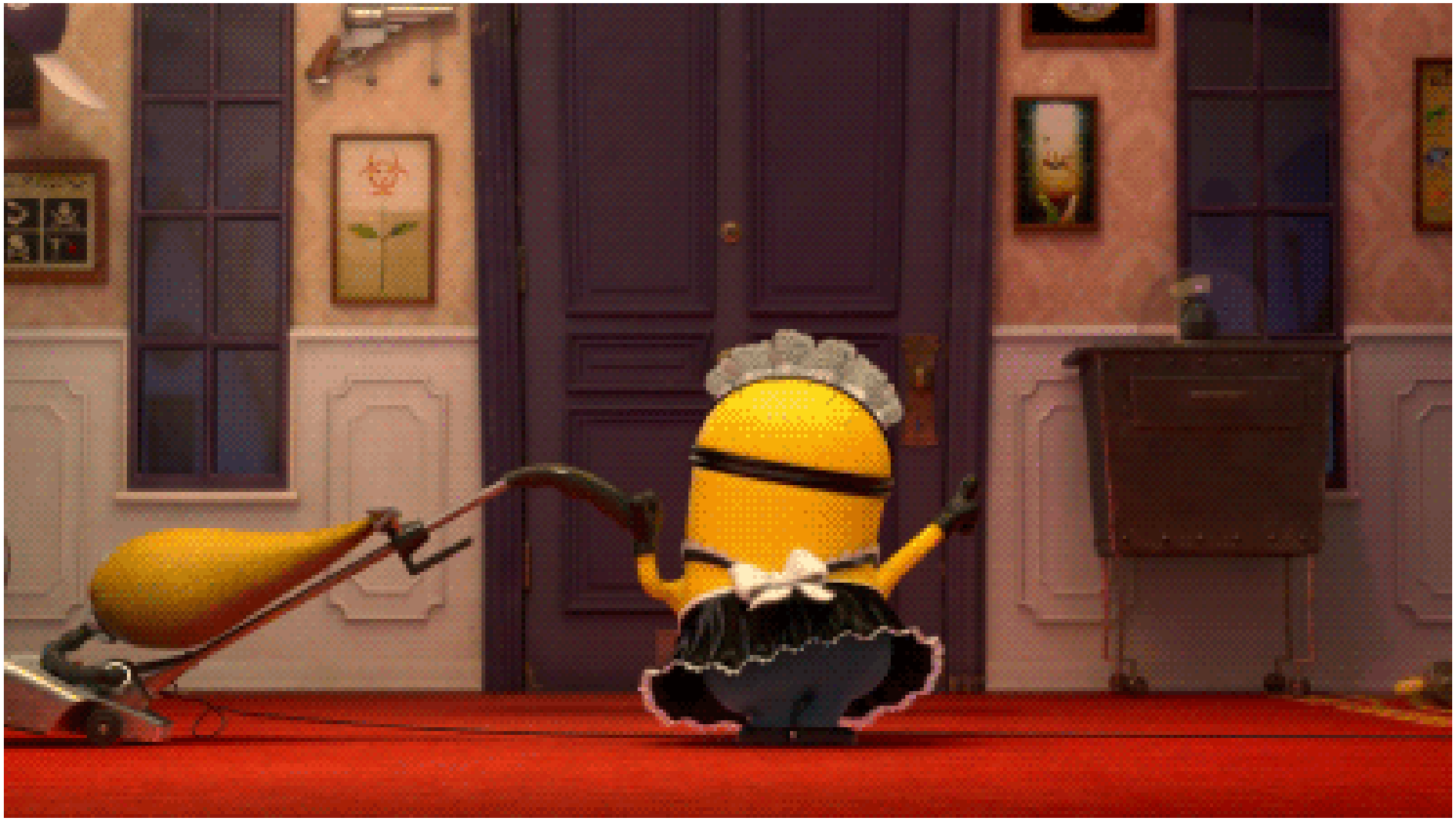
Manipulação de dados



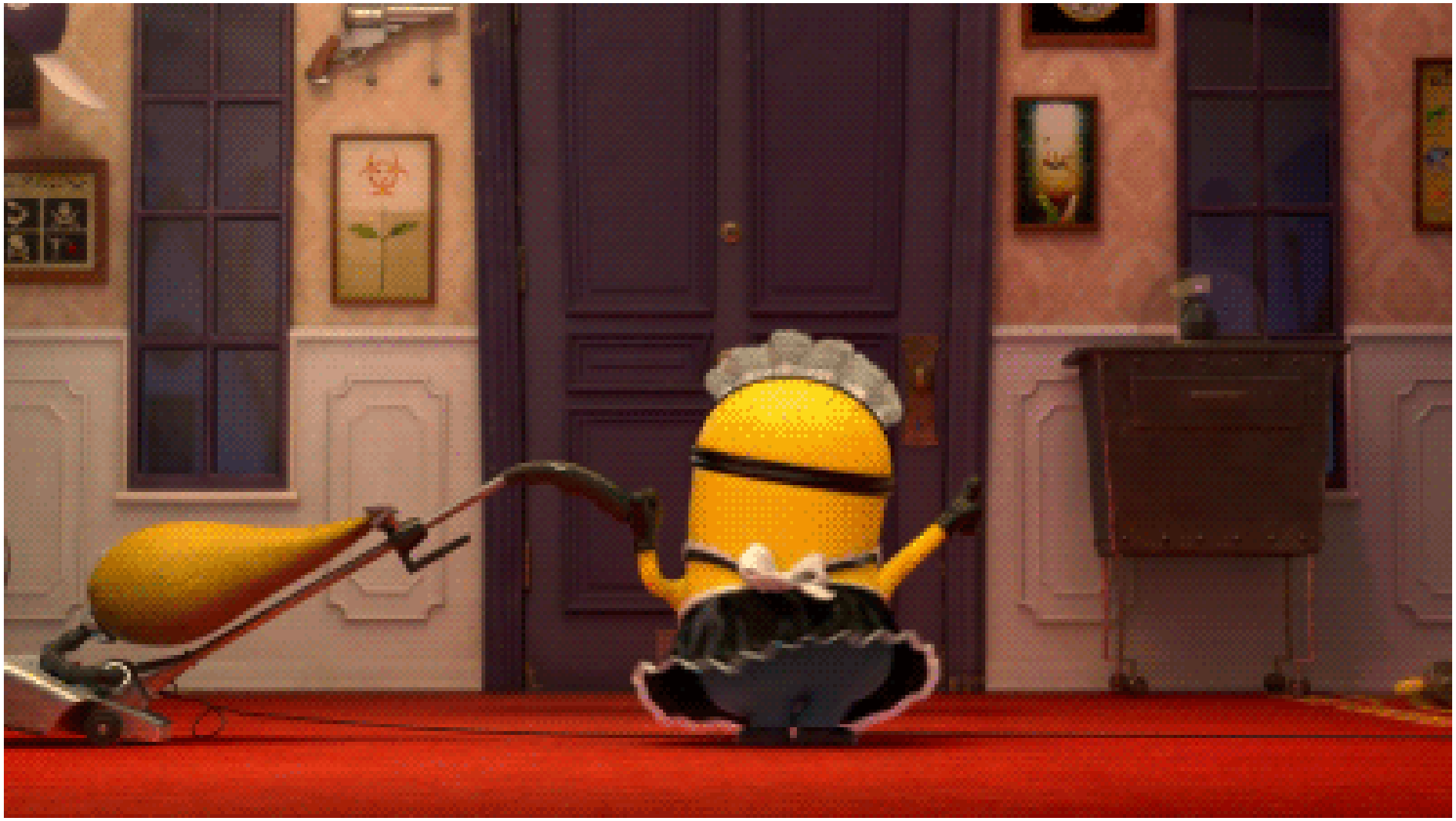
Manipulação de dados



Manipulação de dados



Manipulação de dados

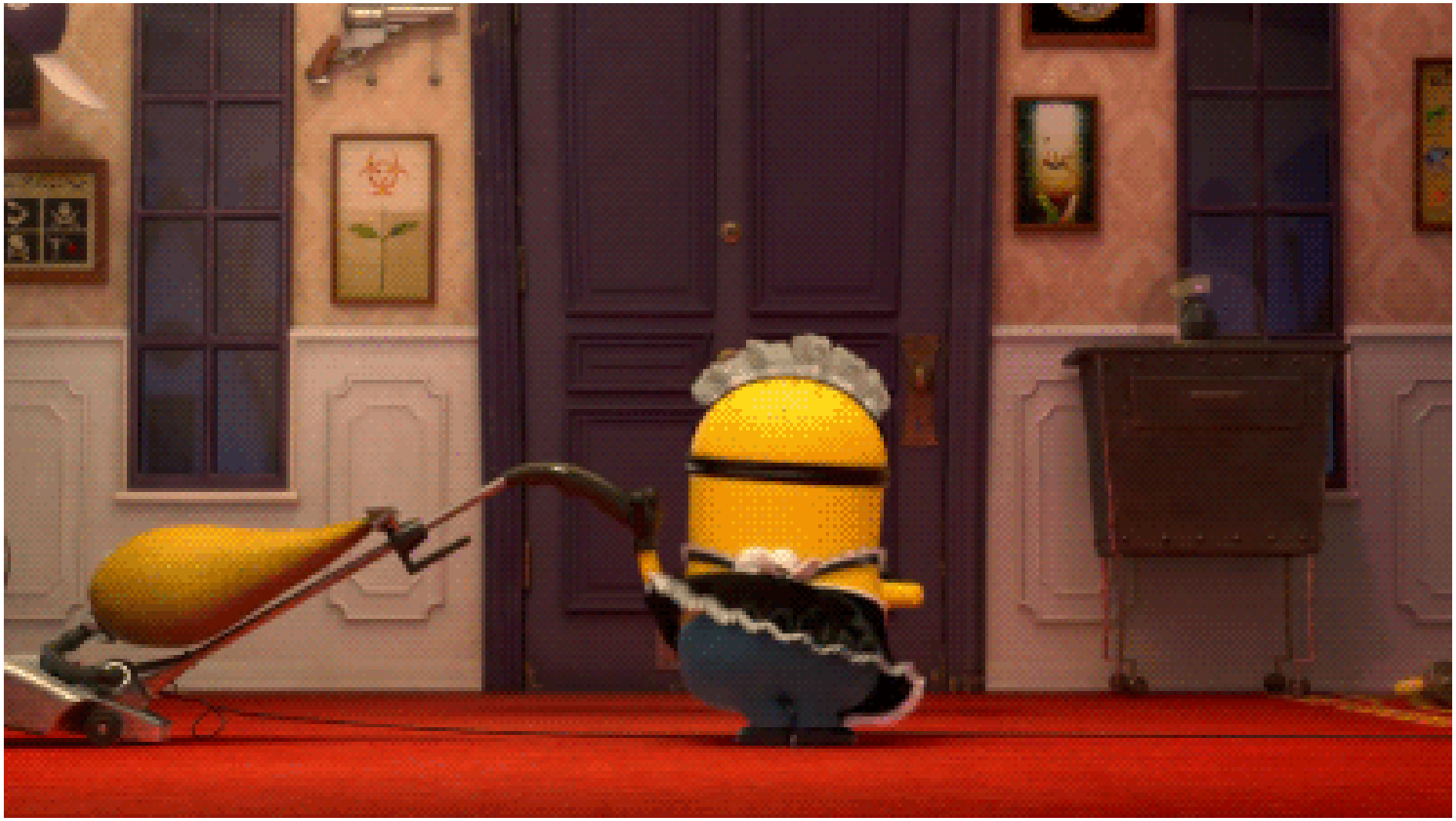


Manipulação de dados

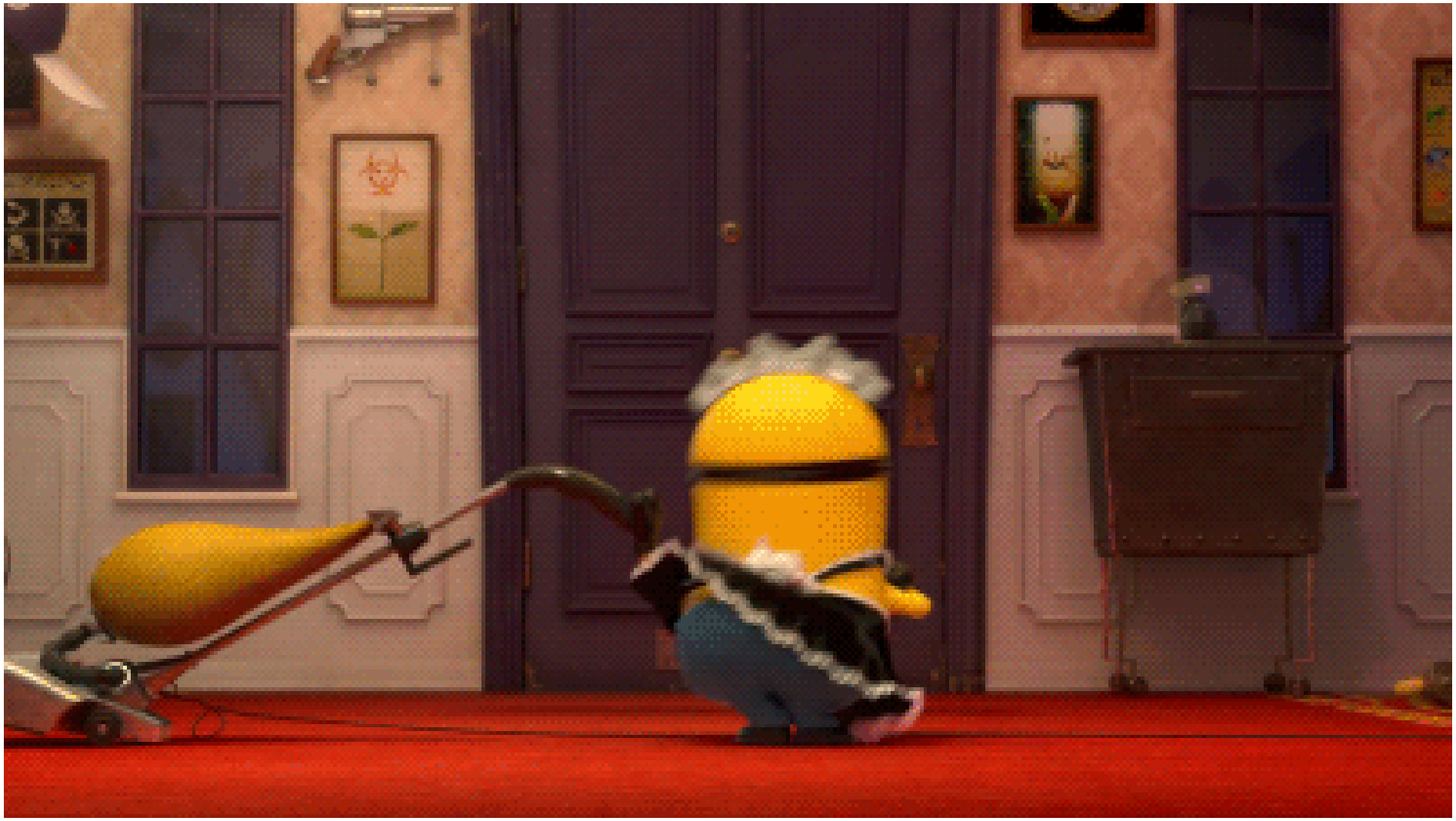
Manipulação de dados

Manipulação de dados

Manipulação de dados



Manipulação de dados



Manipulação de dados

Manipulação de dados

Manipulação de dados

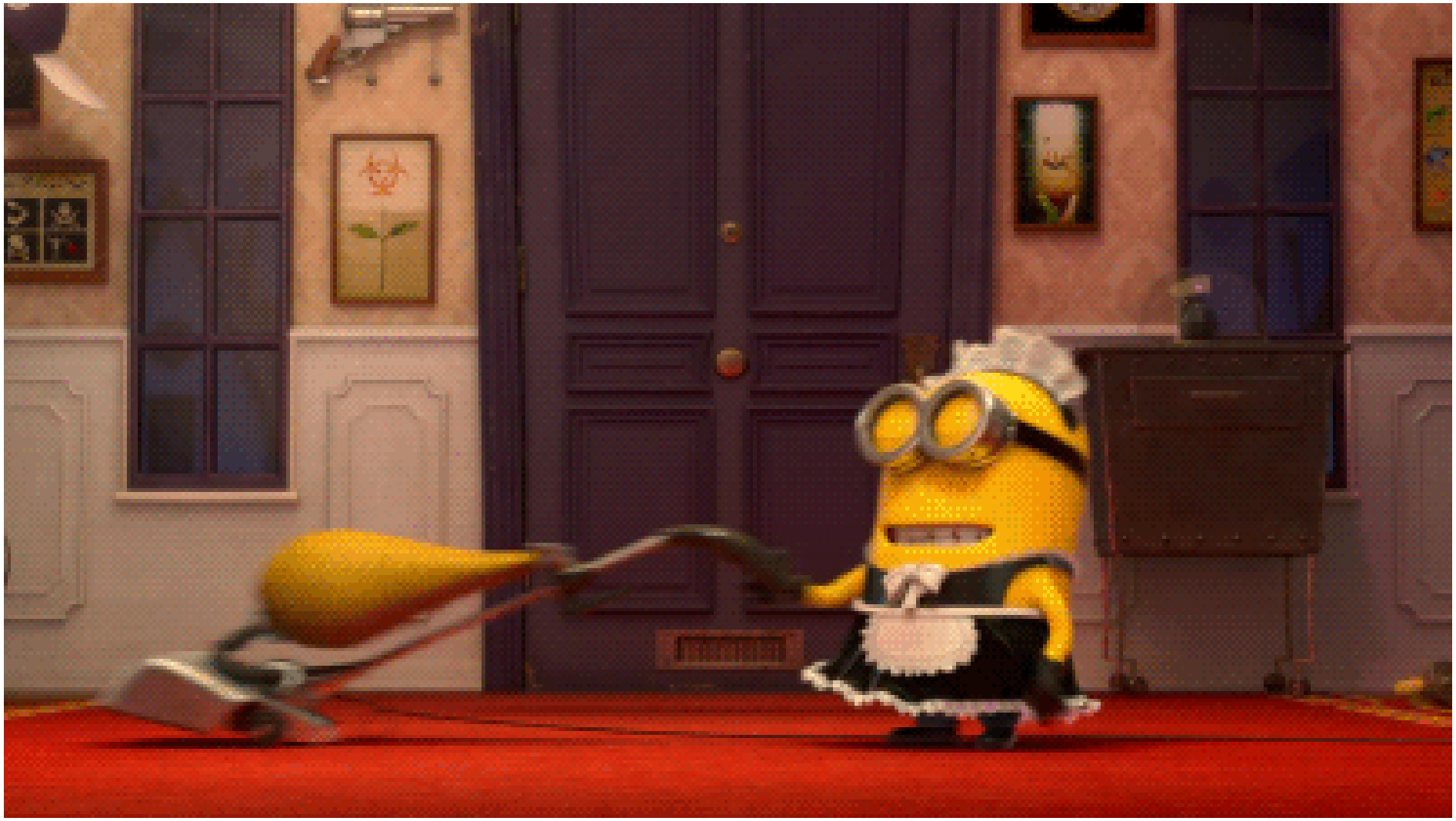
Manipulação de dados

Manipulação de dados

Manipulação de dados

Manipulação de dados

Manipulação de dados



Objectivos

No final deste capítulo, os estudantes deverão saber:

- Organizar os dados no formato desejável;
- Criar novas variávies;
- Filtrar linhas e seleccionar colunas;
- Juntar várias bases de dados numa só base (merge);
- Usar algumas funções da livraria `dplyr` para manipular dados;

Para ilustração de várias funiconalide de manipulação de dados , vamos usar a base de dados `mtcars`.

Suponha que queira aceder as primeiras 10 linhas da base.

```r
data("mtcars")
mtcars[1:10, ]
```

```
##                    mpg cyl  disp  hp drat    wt  qsec vs am
## Mazda RX4         21.0   6 160.0 110 3.90 2.620 16.46  0  1
## Mazda RX4 Wag     21.0   6 160.0 110 3.90 2.875 17.02  0  1
## Datsun 710        22.8   4 108.0  93 3.85 2.320 18.61  1  1
## Hornet 4 Drive    21.4   6 258.0 110 3.08 3.215 19.44  1  0
## Hornet Sportabout 18.7   8 360.0 175 3.15 3.440 17.02  0  0
## Valiant           18.1   6 225.0 105 2.76 3.460 20.22  1  0
## Duster 360        14.3   8 360.0 245 3.21 3.570 15.84  0  0
## Merc 240D         24.4   4 146.7  62 3.69 3.190 20.00  1  0
## Merc 230          22.8   4 140.8  95 3.92 3.150 22.90  1  0
## Merc 280          19.2   6 167.6 123 3.92 3.440 18.30  1  0
```

Pode-se também aceder algumas linhas especificas, por exemplo, suponha que esteja interessado nas linhas 1,3,12,20.

```
mtcars[c(1,3,12,30),]
```

```
##                  mpg cyl  disp  hp drat   wt  qsec vs am gear
## Mazda RX4       21.0   6 160.0 110 3.90 2.62 16.46  0  1    4
## Datsun 710      22.8   4 108.0  93 3.85 2.32 18.61  1  1    4
## Merc 450SE      16.4   8 275.8 180 3.07 4.07 17.40  0  0    3
## Ferrari Dino 19.7   6 145.0 175 3.62 2.77 15.50  0  1    5
```

As primeiras 10 linhas podem ser armazenadas numa variável para uso futuro.

```
prim_10=mtcars[1:10, ]
```

Por vezes, o interesse é verificar linhas para a qual uma dada coluna contem um valor y. Por ex, podemos estar interessado em saber, quais os carros com quatro velocidades. A variável que nos dá informação sobre velocidades é `gear`

```r
mtcars[mtcars$gear==4,]
```

```
##                  mpg cyl  disp  hp drat    wt  qsec vs am ge
## Mazda RX4       21.0   6 160.0 110 3.90 2.620 16.46  0  1
## Mazda RX4 Wag   21.0   6 160.0 110 3.90 2.875 17.02  0  1
## Datsun 710      22.8   4 108.0  93 3.85 2.320 18.61  1  1
## Merc 240D       24.4   4 146.7  62 3.69 3.190 20.00  1  0
## Merc 230        22.8   4 140.8  95 3.92 3.150 22.90  1  0
## Merc 280        19.2   6 167.6 123 3.92 3.440 18.30  1  0
## Merc 280C       17.8   6 167.6 123 3.92 3.440 18.90  1  0
## Fiat 128        32.4   4  78.7  66 4.08 2.200 19.47  1  1
## Honda Civic     30.4   4  75.7  52 4.93 1.615 18.52  1  1
## Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90  1  1
## Fiat X1-9       27.3   4  79.0  66 4.08 1.935 18.90  1  1
## Volvo 142E      21.4   4 121.0 109 4.11 2.780 18.60  1  1
```

Caso o interesse seja conhecer as viaturas com 4 velocidades e 2 carburadores, usaríamos as variáveis `gear` e `carb` para aceder essa informação.

```
mtcars[mtcars$gear==4 & mtcars$carb==2, ]
```

```
##                mpg cyl  disp  hp drat    wt  qsec vs am gear
## Merc 240D     24.4   4 146.7  62 3.69 3.190 20.00  1  0    4
## Merc 230      22.8   4 140.8  95 3.92 3.150 22.90  1  0    4
## Honda Civic   30.4   4  75.7  52 4.93 1.615 18.52  1  1    4
## Volvo 142E    21.4   4 121.0 109 4.11 2.780 18.60  1  1    4
```

Ordenção

O R permite ordenar observações (linhas) de forma crescente e decrescente. O ordenamento é com base na função `order()`.

```
#ordenar por mpg
nova_base=mtcars[order(mtcars$mpg),]
head(nova_base)
```

```
##                      mpg cyl disp  hp drat    wt  qsec vs a
## Cadillac Fleetwood  10.4   8  472 205 2.93 5.250 17.98  0
## Lincoln Continental 10.4   8  460 215 3.00 5.424 17.82  0
## Camaro Z28          13.3   8  350 245 3.73 3.840 15.41  0
## Duster 360          14.3   8  360 245 3.21 3.570 15.84  0
## Chrysler Imperial   14.7   8  440 230 3.23 5.345 17.42  0
## Maserati Bora       15.0   8  301 335 3.54 3.570 14.60  0
```

Pode-se também ordenar usando mais de uma variável.

```r
# Vamos ordenar os dados usando as variáveis mpg e cly

nova_base_1=mtcars[order(mtcars$mpg, mtcars$cyl),]
head(nova_base_1)
```

```
##                      mpg cyl disp  hp drat    wt  qsec vs a
## Cadillac Fleetwood  10.4   8  472 205 2.93 5.250 17.98  0
## Lincoln Continental 10.4   8  460 215 3.00 5.424 17.82  0
## Camaro Z28          13.3   8  350 245 3.73 3.840 15.41  0
## Duster 360          14.3   8  360 245 3.21 3.570 15.84  0
## Chrysler Imperial   14.7   8  440 230 3.23 5.345 17.42  0
## Maserati Bora       15.0   8  301 335 3.54 3.570 14.60  0
```

Caso queira ordenar por ordem decrescente, basta preceder a variável pelo sinal menos(-)

```
base_descr=mtcars[order(-mtcars$mpg),]
head(base_descr)
```

```
##                  mpg cyl  disp  hp drat    wt  qsec vs am ge
## Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90  1  1
## Fiat 128       32.4   4  78.7  66 4.08 2.200 19.47  1  1
## Honda Civic    30.4   4  75.7  52 4.93 1.615 18.52  1  1
## Lotus Europa   30.4   4  95.1 113 3.77 1.513 16.90  1  1
## Fiat X1-9      27.3   4  79.0  66 4.08 1.935 18.90  1  1
## Porsche 914-2  26.0   4 120.3  91 4.43 2.140 16.70  0  1
```

Selecção de variáveis

O R também permite fazer selecção de variáveis. Este procedimento é importante quando se pretende trabalhar com um número reduzido de variáveis.

```
#suponha que queira apenas trabalhar com as variáveis mpg, cyl

cols=c('mpg','cyl','drat') # criacao de vector com as variáve
sel=mtcars[, cols]
head(sel)
```

```
##                    mpg cyl drat
## Mazda RX4         21.0   6 3.90
## Mazda RX4 Wag     21.0   6 3.90
## Datsun 710        22.8   4 3.85
## Hornet 4 Drive    21.4   6 3.08
## Hornet Sportabout 18.7   8 3.15
## Valiant           18.1   6 2.76
```

Para além de usar os nomes das variáveis, pode-se usar o número da coluna de cada variável para seleccionar as varíaveis de interesse.

```
sel_num=mtcars[, c(1,2,3,10)]
head(sel_num)
```

```
##                    mpg cyl disp gear
## Mazda RX4         21.0   6  160    4
## Mazda RX4 Wag     21.0   6  160    4
## Datsun 710        22.8   4  108    4
## Hornet 4 Drive    21.4   6  258    3
## Hornet Sportabout 18.7   8  360    3
## Valiant           18.1   6  225    3
```

Exclusão de variáveis.

```r
del_var=mtcars[,c(-3,-5,-9)]
head(del_var)
```

```
##                    mpg cyl  hp    wt  qsec vs gear carb
## Mazda RX4         21.0   6 110 2.620 16.46  0    4    4
## Mazda RX4 Wag     21.0   6 110 2.875 17.02  0    4    4
## Datsun 710        22.8   4  93 2.320 18.61  1    4    1
## Hornet 4 Drive    21.4   6 110 3.215 19.44  1    3    1
## Hornet Sportabout 18.7   8 175 3.440 17.02  0    3    2
## Valiant           18.1   6 105 3.460 20.22  1    3    1
```

O procedimento anterior deixa a base de dados original intacta. Mas caso queira remover(apagar) por definitivo uma determinada variável (que é pouco recomendado) pode-se fazer da seguinte maneira:

```
#vamos apagar a variável mpg da base de dados.4
mtcars$mpg=NULL
head(mtcars)
```

```
##                   cyl disp  hp drat    wt  qsec vs am gear
## Mazda RX4           6  160 110 3.90 2.620 16.46  0  1    4
## Mazda RX4 Wag       6  160 110 3.90 2.875 17.02  0  1    4
## Datsun 710          4  108  93 3.85 2.320 18.61  1  1    4
## Hornet 4 Drive      6  258 110 3.08 3.215 19.44  1  0    3
## Hornet Sportabout   8  360 175 3.15 3.440 17.02  0  0    3
## Valiant             6  225 105 2.76 3.460 20.22  1  0    3
```

Filtro e seleçcão de colunas em simultâneo

Até aqui vimos como fazer filtros e seleccionar colunas de formas separadas. Pode-se muito bem fazer os dois procedimentos em simultâneo.

```
#viaturas com mais de 4 cilindros (apresentar penas  a coluna
data(mtcars)
mtcars[mtcars$cyl>4, c(8,9)]
```

```
##                     vs am
## Mazda RX4            0  1
## Mazda RX4 Wag        0  1
## Hornet 4 Drive       1  0
## Hornet Sportabout    0  0
## Valiant              1  0
## Duster 360           0  0
## Merc 280             1  0
## Merc 280C            1  0
## Merc 450SE           0  0
## Merc 450SL           0  0
```

Combinação de base de dados

Para a combinação de dados iremos usar as seguintes funções:

- `rbind()` para juntar linhas;
- `cbind()` para juntar colunas;
- `merge()` para juntar bases de dados com base numa chave de identificação;
- Algumas funções da livraria `dplyr` para junção de base de dados;

rbind()

A função `rbind()` retorna a junção de linhas de duas bases de dados, que têm mesmas variáveis (colunas).

Name	City	Country
Lenna	San Francisco	US
Malcom	New York	US
Akiko	Tokyo	Japan

+

Name	City	Country
Lenna	San Francisco	US
Thomas	London	UK
Diane	Chicago	US

→

Name	City	Country
Lenna	San Francisco	US
Malcom	New York	US
Akiko	Tokyo	Japan
Lenna	San Francisco	US
Thomas	London	UK
Diane	Chicago	US

```r
data_1=data.frame(Name=c('Lenna','Malcom',"Akiko"),
                  City=c("San Francisco","New York", "Tokyo"),
data_2=data.frame(Name=c("Lenna","Thomas","Diane"),
                  City=c("San Franscico", "London", "Chicago")
data_1
```

```
##     Name          City Country
## 1  Lenna San Francisco      US
## 2 Malcom      New York      US
## 3  Akiko         Tokyo   Japan
```

```r
data_2
```

```
##     Name          City Country
## 1  Lenna San Franscico      US
## 2 Thomas        London      UK
## 3  Diane       Chicago      US
```

```r
data_new=rbind(data_1, data_2)
data_new
```

```
##     Name          City Country
## 1  Lenna San Francisco      US
## 2 Malcom      New York      US
## 3  Akiko         Tokyo   Japan
## 4  Lenna San Franscico      US
## 5 Thomas        London      UK
## 6  Diane       Chicago      US
```

cbind()

Retorna a junção de colunas de duas bases de dados, quando estas têm o mesmo número de linhas.

Data Frame 1

	A	B
Row 1	1	4
Row 2	3	2
Row 3	7	5
Row 4	8	9

Data Frame 2

	C	D
Row 1	9	12
Row 2	2	12
Row 3	4	15
Row 4	8	4

cbind

New Data Frame

	A	B	C	D
Row 1	1	4	9	12
Row 2	3	2	2	12
Row 3	7	5	4	15
Row 4	8	9	8	4

```r
data_3=data.frame(Age=c(14,16,17,18,19,20))
data_3
```

```
##   Age
## 1  14
## 2  16
## 3  17
## 4  18
## 5  19
## 6  20
```

```r
data_4=cbind(data_new, data_3)
data_4
```

```
##     Name          City Country Age
## 1  Lenna San Francisco      US  14
## 2 Malcom      New York      US  16
## 3  Akiko         Tokyo   Japan  17
## 4  Lenna San Franscico      US  18
```

Merge/Join

- Merge permite unir duas bases dados com base numa identificação(ID).
- Existem vários tipos de merge: natural join, left join e right outer join

natural join/inner join

Employee ID	First Name	Department ID
1	John	10
2	Daniel	20
3	Anne	10
4	George	20
5	Tim	10

Department ID	Department Name
10	Sales
20	HR

First Name	Department Name
John	Sales
Daniel	HR
Anne	Sales
George	HR
Tim	Sales

```r
df1=data.frame(employee_ID=c(1,2,3,4,5),first_name=c("Jone","Daniel","Anne","George","Tim"),
               department_ID=c(10,20,10,20,10))
df2=data.frame(department_ID=c(10,20),department_name=c('sales', 'HR'))
df_all=merge(df1,df2, by.x='department_ID',by.y='department_ID')
df_all
```

```
##   department_ID employee_ID first_name department_name
## 1            10           1       Jone           sales
## 2            10           3       Anne           sales
## 3            10           5        Tim           sales
## 4            20           2     Daniel              HR
## 5            20           4     George              HR
```

Left Join

Retorna todas as linhas da tabela da esquerda e todas as linhas com as chaves correspondentes da tabela da direita.

```
df1 = data.frame(CustomerId = c(1:6),
                 Product = c(rep("Oven", 3), rep("Television", 3)))
df1
```

```
##   CustomerId    Product
## 1          1       Oven
## 2          2       Oven
## 3          3       Oven
## 4          4 Television
## 5          5 Television
## 6          6 Television
```

```r
# data frame 2
df2 = data.frame(CustomerId = c(2, 4, 6),
                 State = c(rep("California", 2), rep("Texas",
df2
```

```
##   CustomerId      State
## 1          2 California
## 2          4 California
## 3          6      Texas
```

```r
df<-merge(x=df1,y=df2,by="CustomerId",all.x=TRUE)
df
```

```
##   CustomerId    Product      State
## 1          1       Oven       <NA>
## 2          2       Oven California
## 3          3       Oven       <NA>
## 4          4 Television California
## 5          5 Television       <NA>
## 6          6 Television      Texas
```

Right outer join

Retorna todas as linhas da tabela da direita e todas as linhas com as chaves correspondentes da tabela da esquerda.

```
df<-merge(x=df1,y=df2,by="CustomerId",all.y=TRUE)
df
```

```
##   CustomerId    Product      State
## 1          2       Oven California
## 2          4 Television California
## 3          6 Television      Texas
```

dplyr

- Quase que todos os pacotes que implementam modelos estatísticos funcionam em data frame.
- Manipulação de dados usando *"built-in function"* exige mais programação do que análise de dados.
- Com a criação do pacote "dplyr' o processo de manipulação de dados tornou-se muito mais fácil.

select()

```r
# isntall.packages(dplyr)
library(dplyr)
iris %>% tbl_df %>% select(Petal.Width) %>% head(3)
```

```
## # A tibble: 3 x 1
##   Petal.Width
##         <dbl>
## 1         0.2
## 2         0.2
## 3         0.2
```

Pode seleccionar, por exemplo, mais do que uma coluna.

```
# removi tbl_df
iris %>%
  select(Petal.Width, Sepal.Length, Petal.Length) %>%
  head(3)
```

```
##   Petal.Width Sepal.Length Petal.Length
## 1         0.2          5.1          1.4
## 2         0.2          4.9          1.4
## 3         0.2          4.7          1.3
```

Existem várias formas de seleccionar colunas.

```
iris %>% tbl_df %>%
select(starts_with("Petal")) %>% head(3)
```

```
## # A tibble: 3 x 2
##   Petal.Length Petal.Width
##          <dbl>       <dbl>
## 1          1.4         0.2
## 2          1.4         0.2
## 3          1.3         0.2
```

```r
iris %>% tbl_df %>%
select(ends_with("Width")) %>% head(3)
## # A tibble: 3 × 2
## Sepal.Width Petal.Width
## <dbl> <dbl>
## 1 3.5 0.2
## 2 3.0 0.2
## 3 3.2 0.2
iris %>% tbl_df %>%
select(contains("etal")) %>% head(3)
## # A tibble: 3 × 2
## Petal.Length Petal.Width
## <dbl> <dbl>
## 1 1.4 0.2
## 2 1.4 0.2
## 3 1.3 0.2
```

mutate()

A função `mutate()` permite adicionar uma coluna ao seu quadro de dados, especificando uma expressão de como calculá-la:

```
iris %>% tbl_df %>%
mutate(Petal.Width.plus.Length = Petal.Width + Petal.Length) %
select(Species, Petal.Width.plus.Length) %>%
head(3)
```

```
## # A tibble: 3 x 2
##   Species Petal.Width.plus.Length
##   <fct>                     <dbl>
## 1 setosa                     1.60
## 2 setosa                     1.60
## 3 setosa                     1.5
```

arrange()

Ordena uma base de dados com base numa variável.

```
iris %>% tbl_df %>%
arrange(Sepal.Length)
```

```
## # A tibble: 150 x 5
##    Sepal.Length Sepal.Width Petal.Length Petal.Width Specie
##           <dbl>       <dbl>        <dbl>       <dbl> <fct>
##  1          4.3         3            1.1         0.1 setosa
##  2          4.4         2.9          1.4         0.2 setosa
##  3          4.4         3            1.3         0.2 setosa
##  4          4.4         3.2          1.3         0.2 setosa
##  5          4.5         2.3          1.3         0.3 setosa
##  6          4.6         3.1          1.5         0.2 setosa
##  7          4.6         3.4          1.4         0.3 setosa
##  8          4.6         3.6          1           0.2 setosa
##  9          4.6         3.2          1.4         0.2 setosa
## 10          4.7         3.2          1.3         0.2 setosa
```

Por defeito ela ordena por odem crescente, mas pode-se ordenar por ordem decrescente usandoa função `desc()`

```
iris %>% tbl_df %>%
arrange(desc(Sepal.Length))
```

```
## # A tibble: 150 x 5
##    Sepal.Length Sepal.Width Petal.Length Petal.Width Specie
##           <dbl>       <dbl>        <dbl>       <dbl> <fct>
##  1          7.9         3.8          6.4         2   virgin
##  2          7.7         3.8          6.7         2.2 virgin
##  3          7.7         2.6          6.9         2.3 virgin
##  4          7.7         2.8          6.7         2   virgin
##  5          7.7         3            6.1         2.3 virgin
##  6          7.6         3            6.6         2.1 virgin
##  7          7.4         2.8          6.1         1.9 virgin
##  8          7.3         2.9          6.3         1.8 virgin
##  9          7.2         3.6          6.1         2.5 virgin
## 10          7.2         3.2          6           1.8 virgin
## #     with 140 more rows
```

filter()

A função `filter()` permite selecionar linhas com base em expressões lógicas. Você dá à função um predicado, especificando o que uma linha deve satisfazer para ser incluída.

```
iris %>% tbl_df %>%
filter(Sepal.Length > 5)
```

```
## # A tibble: 118 x 5
##    Sepal.Length Sepal.Width Petal.Length Petal.Width Specie
##           <dbl>       <dbl>        <dbl>       <dbl> <fct>
##  1          5.1         3.5          1.4         0.2 setosa
##  2          5.4         3.9          1.7         0.4 setosa
##  3          5.4         3.7          1.5         0.2 setosa
##  4          5.8         4            1.2         0.2 setosa
##  5          5.7         4.4          1.5         0.4 setosa
##  6          5.4         3.9          1.3         0.4 setosa
##  7          5.1         3.5          1.4         0.3 setosa
##  8          5.7         3.8          1.7         0.3 setosa
```

Pode-se ter mais do que uma expressão lógica.

```
iris %>% tbl_df %>%
filter(Sepal.Length > 5 & Species == "virginica") %>%
select(Species, Sepal.Length) %>%
head(3)
```

```
## # A tibble: 3 x 2
##   Species   Sepal.Length
##   <fct>            <dbl>
## 1 virginica          6.3
## 2 virginica          5.8
## 3 virginica          7.1
```